

DIJKSTRA'S ALGORITHM

SHORTEST PATH
ROUTING

**Puzzolante - Scarale S. - Bisceglia - De Bonis -
Cocomazzi - Tancredi - Cisternino - Lauriola -
Vigilante.**

DI COSA PARLEREMO?

1. Introduzione:

- a. Grafo – Ricerca percorso minimo – Divisione algoritmi routing;

2. Obiettivi:

- a. Trovare il percorso più breve - Garantire l'ottimalità - Trovare tutti i percorsi più brevi;

3. Funzionamento algoritmo:

- a. Animazione con annessa spiegazione;

4. Pro:

- a. Efficienza - Precisione - Facilità;

5. Contro:

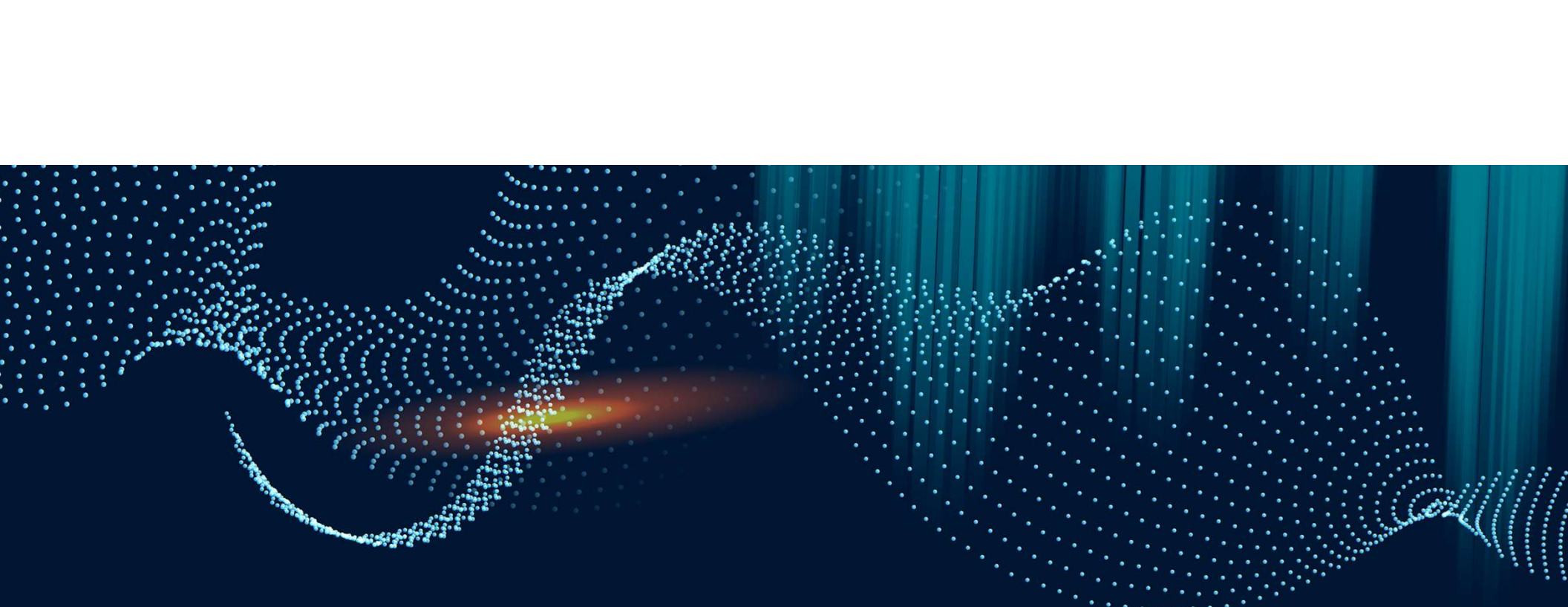
- a. Complessità spaziale - Inefficacia in grafi con cicli negativo - Complessità computazionale
- Limitazioni nella gestione di più percorsi.

6. Approfondimento:

- a. Pathfinding e navigazione dei personaggi – Pianificazione di missioni e obiettivi –
Creazioni di mappe e ambienti di gioco – Ottimizzazione delle prestazioni.

7. Conclusioni:

- a. Efficienza e versatilità – Facilmente implementabile – Limiti.



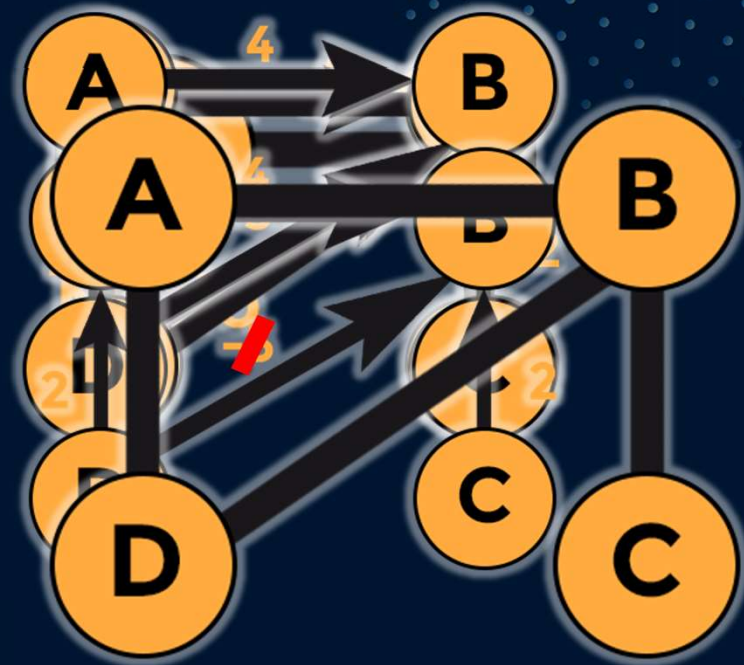
01

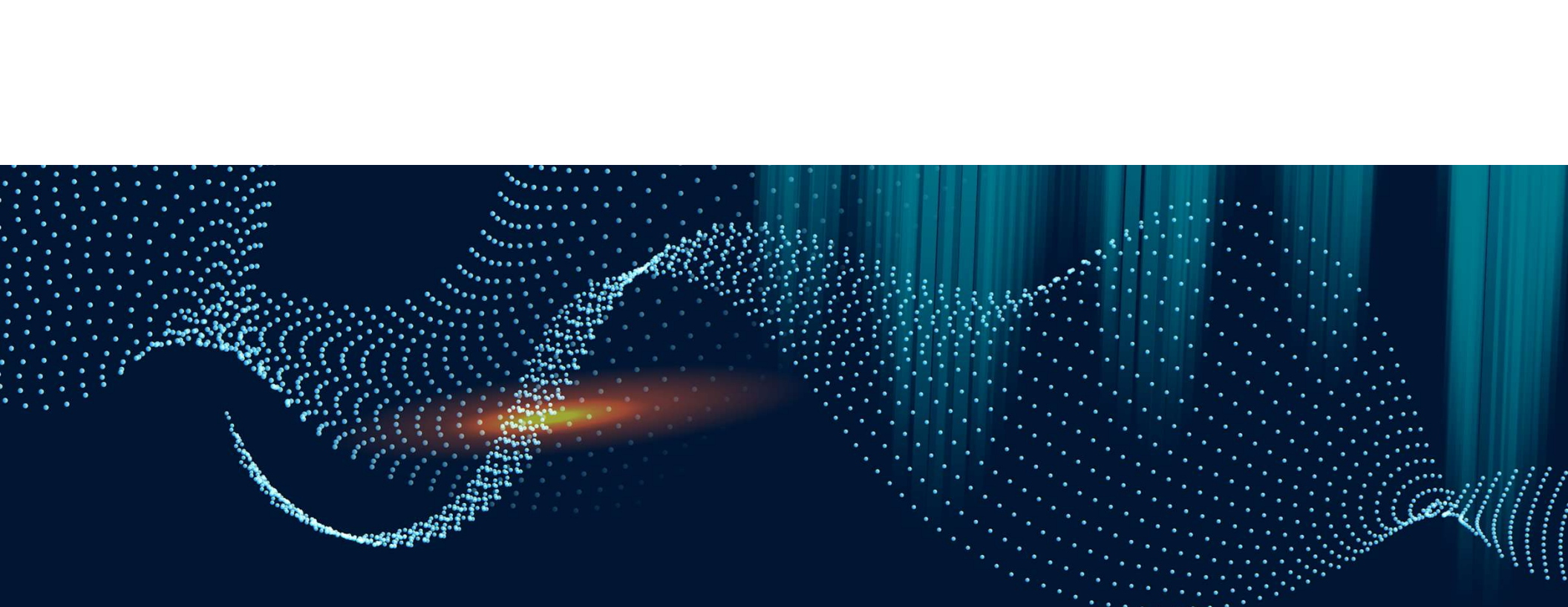
Introduzione

Parliamo dell' algoritmo di Dijkstra.

Introduzione

- **Grafo:**
 - Nodi e archi;
 - Grafo orientato, non orientato;
 - Grafo pesato, non pesato;
- **Ricerca percorso minimo:**
 - Pesi non negativi;
- **Divisione algoritmi routing:**
 - Topologia:
 - Algoritmi distribuiti;
 - Algoritmi isolati;
 - Algoritmi centralizzati;
 - Adattabilità:
 - Statici;
 - Dinamici;





02

Obiettivi

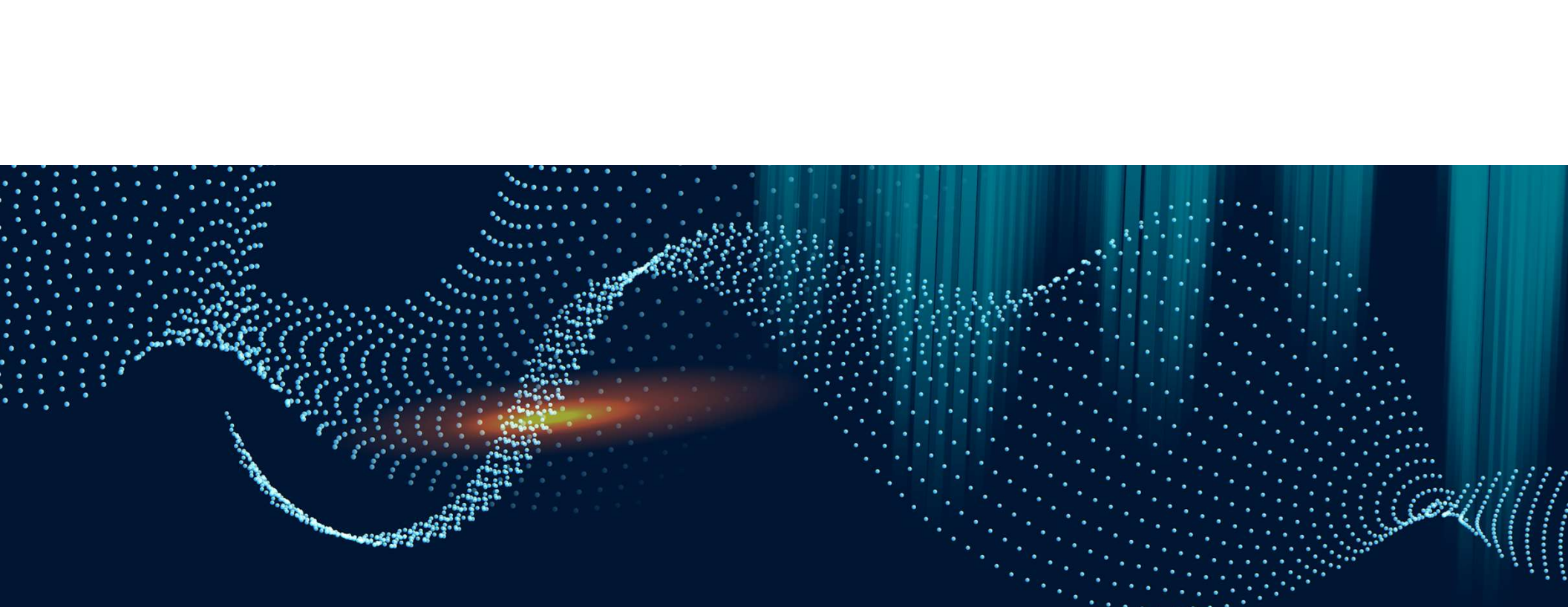
Quali sono gli obiettivi prefissati?

Obiettivi

- **Trovare il percorso più breve:**
 - Da un nodo sorgente a una destinazione desiderata all'interno di un grafo, determinato sommando i pesi degli archi nel grafo;
- **Garantire l'ottimalità:**
 - Garantisce che il percorso trovato sia il più breve possibile, assumendo che i pesi degli archi siano non negativi; questo significa che, se esiste un percorso più breve, l'algoritmo lo troverà;

Obiettivi

- **Trovare tutti i percorsi più brevi:**
 - L'algoritmo di Dijkstra può essere adattato per trovare tutti percorsi più brevi da un nodo sorgente a tutti gli altri nodi nel grafo; ciò è utile, ad esempio nel routing di rete o nella pianificazione dei trasporti. Infatti questo processo continua fino a quando non si è esplorato tutti i nodi raggiungibili dal nodo sorgente o fino a quando non si raggiunge la destinazione desiderata.

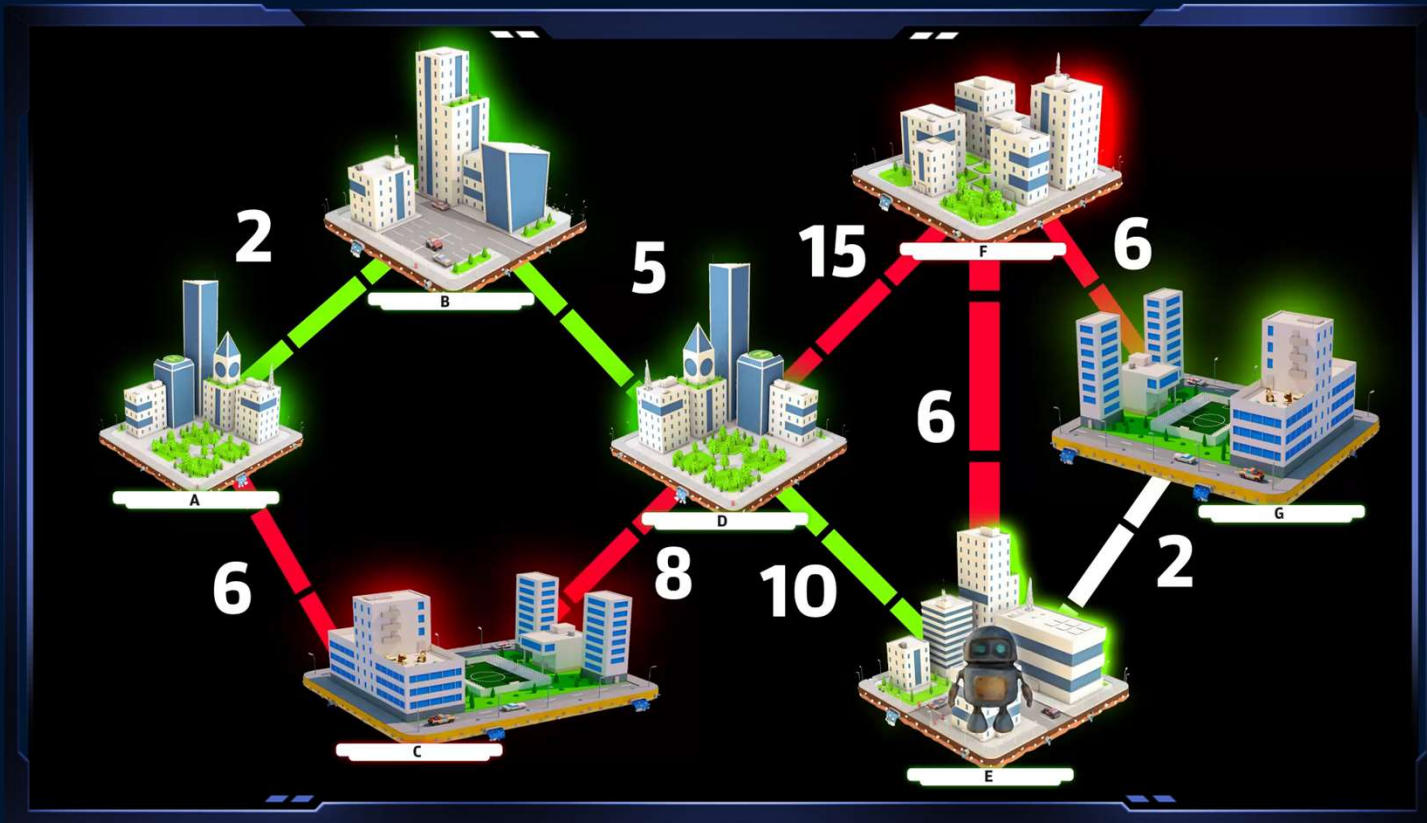


03

Funzionamento

Come funziona?

Funzionamento



Distanza:

A	0	
B	∞	2
C	∞	6
D	∞	7
E	∞	17
F	∞	22
G	∞	19

Alla ricerca dei nodi: { A, B ,C ,D, E ,F ,G }



04

Pro

Principali vantaggi

Pro

- **Efficienza:**

- L'algoritmo di Dijkstra è noto per la sua efficienza computazionale, specialmente su grafi sparsi. Questo lo rende adatto per l'uso in applicazioni in tempo reale e su grandi grafi.

- **Precisione:**

- L'algoritmo di Dijkstra trova sempre il percorso più breve tra due nodi in un grafo con pesi sugli archi non negativi. Non fa supposizioni o approssimazioni sul grafo o sui pesi degli archi, garantendo la precisione del risultato.

Pro

- **Facilità di implementazione :**

- Concettualmente, l'algoritmo di Dijkstra è abbastanza semplice da comprendere e implementare. Non richiede conoscenze avanzate di algoritmi complessi e può essere implementato utilizzando strutture dati di base, come array e code di priorità.

- **Adattibilità:**

- L'algoritmo di Dijkstra può essere facilmente adattato per risolvere una varietà di problemi correlati, come il calcolo del cammino più breve tra tutti i vertici in un grafo (algoritmo di Dijkstra per grafi con sorgente multipla) o il calcolo del cammino più breve tra un singolo nodo di partenza e tutti gli altri nodi (algoritmo di Dijkstra con coda di priorità).

- **Applicabilità a grafi orientati e non orientati :**

- L'algoritmo di Dijkstra può essere applicato sia a grafi orientati che a grafi non orientati, rendendolo flessibile e adatto a una vasta gamma di problemi reali.



05 | **Contro**

Principali svantaggi

Contro

- **Complessità spaziale:**

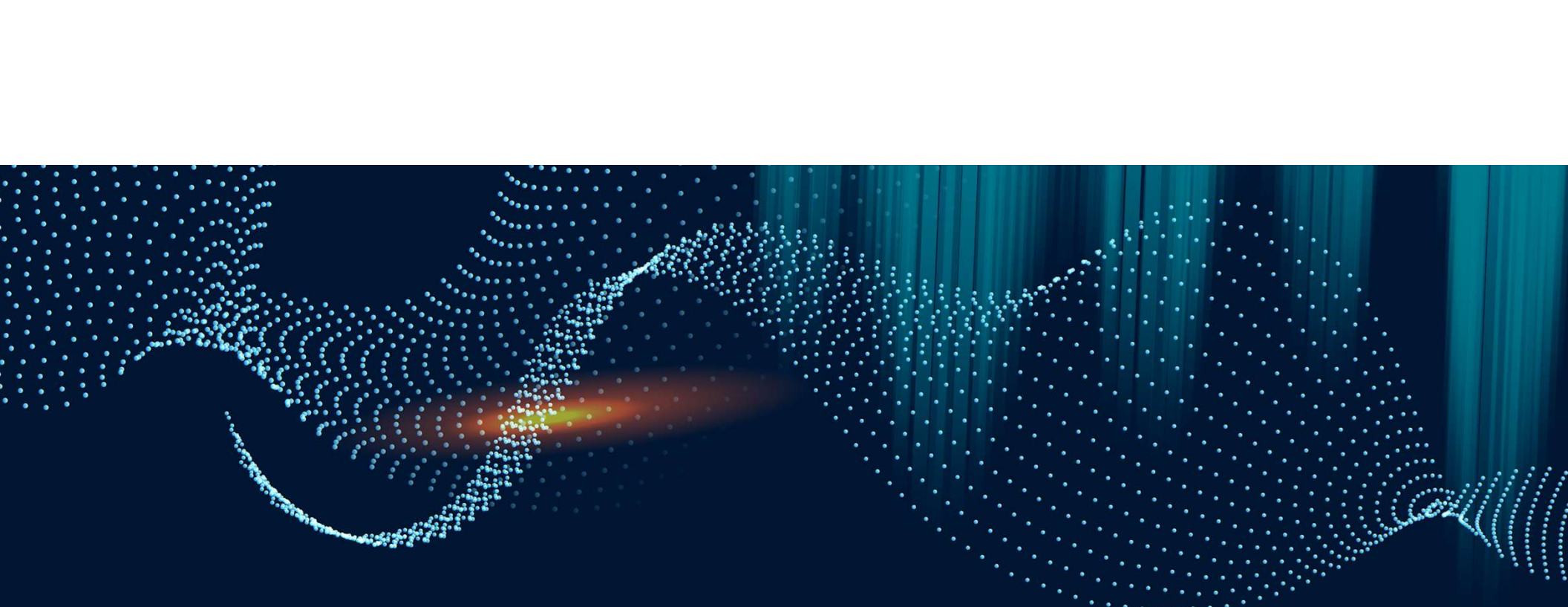
- Anche se Dijkstra è efficiente dal punto di vista temporale, la sua implementazione può richiedere una quantità considerevole di memoria, specialmente per grafi molto grandi. Questo potrebbe essere un problema in contesti in cui la memoria è limitata.

- **Inefficacia in grafi con cicli negativo:**

- Se il grafo contiene cicli di peso negativo, l'algoritmo di Dijkstra potrebbe restare bloccato in un loop infinito o produrre risultati errati. Questo limite impedisce l'utilizzo di Dijkstra in grafi con tale struttura.

Contro

- **Complessità computazionale:**
 - Sebbene l'algoritmo di Dijkstra abbia una complessità temporale efficiente in teoria, potrebbe diventare inefficiente in pratica con grafi molto grandi. Questo può causare ritardi nell'esecuzione o renderlo impraticabile per l'uso in contesti in cui sono richieste risposte immediate.
- **Limitazioni nella gestione di più percorsi:**
 - Dijkstra è progettato per trovare il percorso più breve tra un nodo di partenza e tutti gli altri nodi nel grafo. Tuttavia, se è necessario trovare più percorsi, può essere necessario eseguire l'algoritmo più volte, aumentando il tempo di esecuzione complessivo.



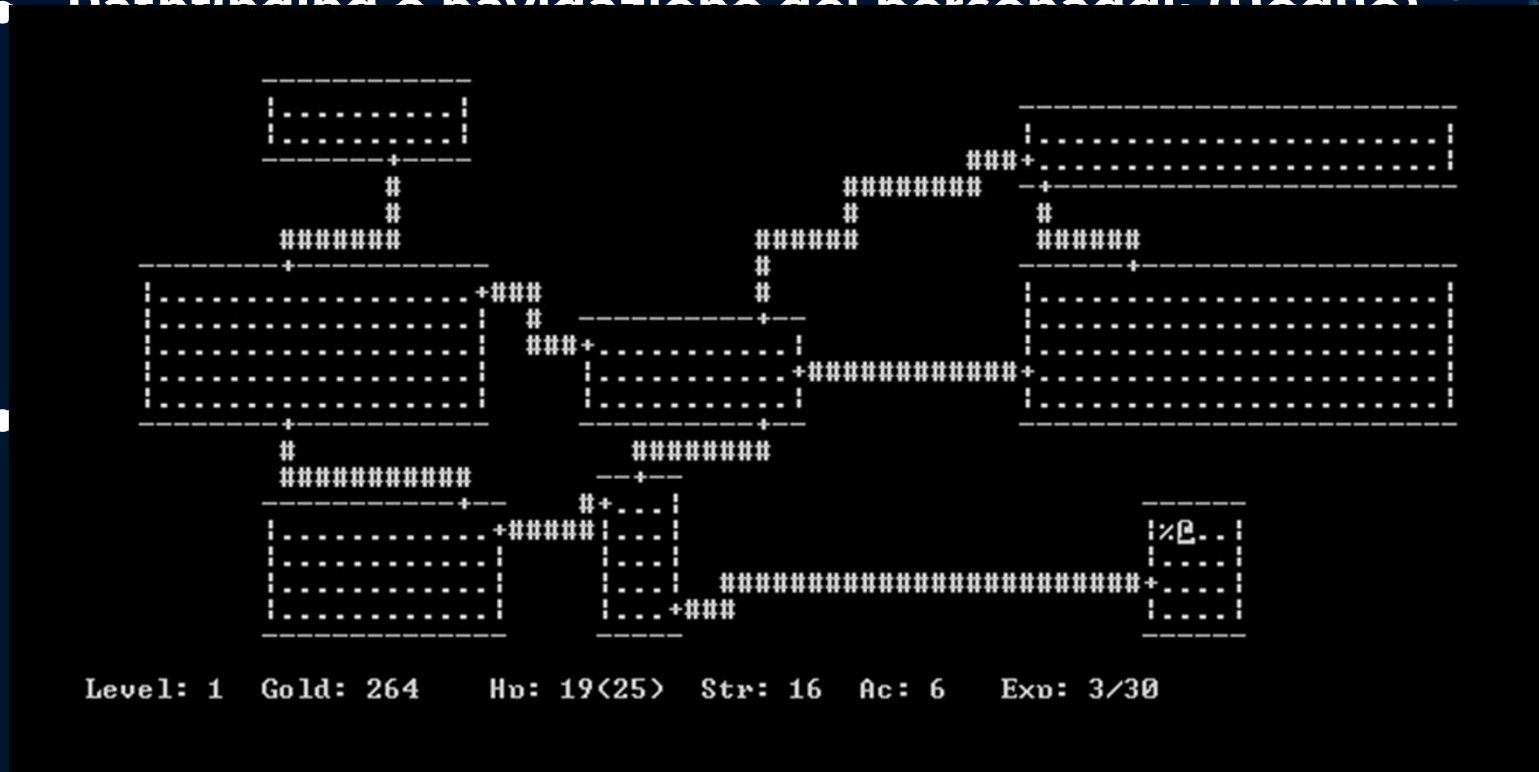
06

Approfondimento

Com'è cambiato il mondo dei giochi.

Approfondimento

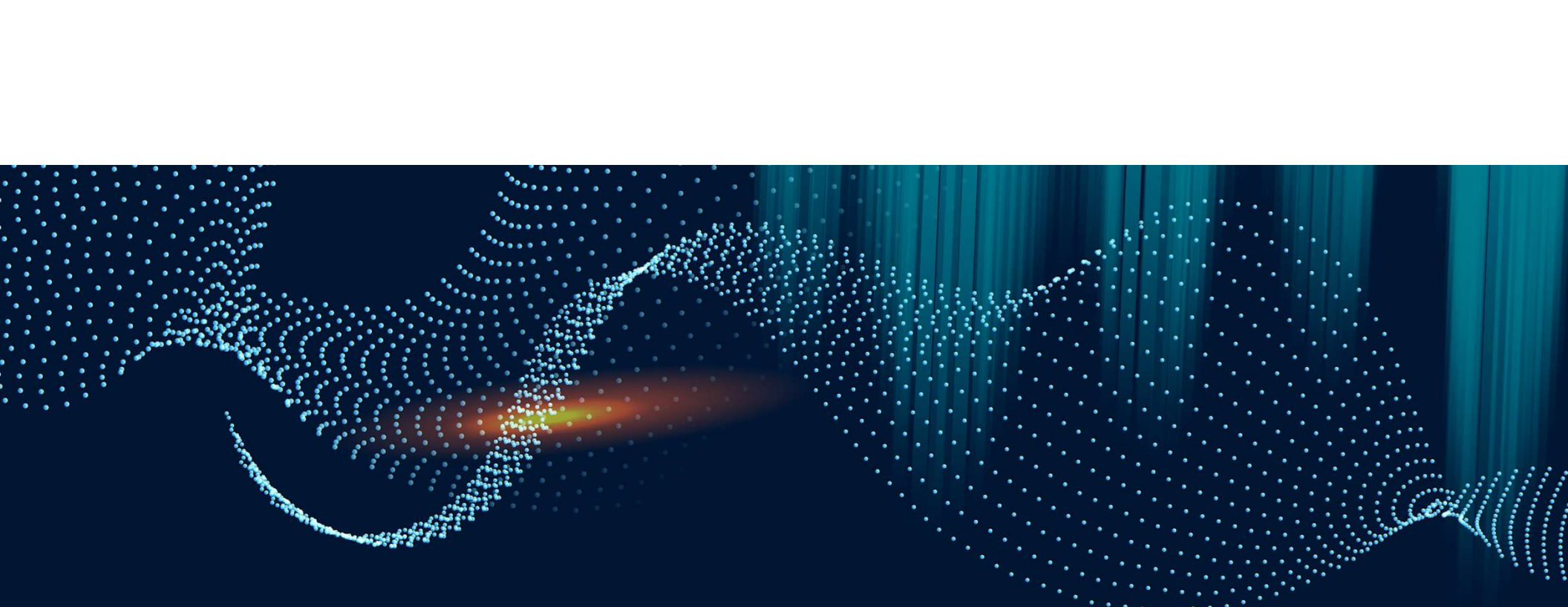
Pathfinding e navigazione dei personaggi (Dogue)



l'esperienza di gioco più convergente e soddisfacente per i giocatori.

Approfondimento

- **Creazione di mappe e ambienti di gioco:**
 - L'algoritmo di Dijkstra può essere utilizzato per la generazione procedurale di mappe e ambienti di gioco, garantendo la connettività e l'accessibilità di diverse aree.
 - Ciò consente di creare mondi di gioco più vasti e complessi, con una maggiore varietà e esplorazione per i giocatori.
- **Ottimizzazione delle prestazioni:**
 - L'algoritmo di Dijkstra, essendo altamente efficiente e ottimizzato, può essere utilizzato per migliorare le prestazioni dei videogiochi, soprattutto in scenari con numerosi personaggi e oggetti in movimento.
 - Ciò consente di mantenere un'esperienza di gioco fluida e reattiva, anche su hardware meno potenti.



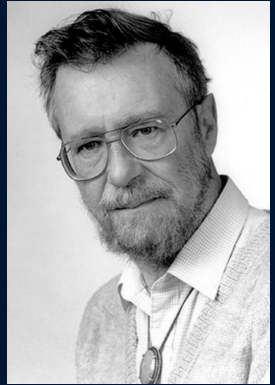
07 | **Conclusioni**

Riassunto

Conclusioni:

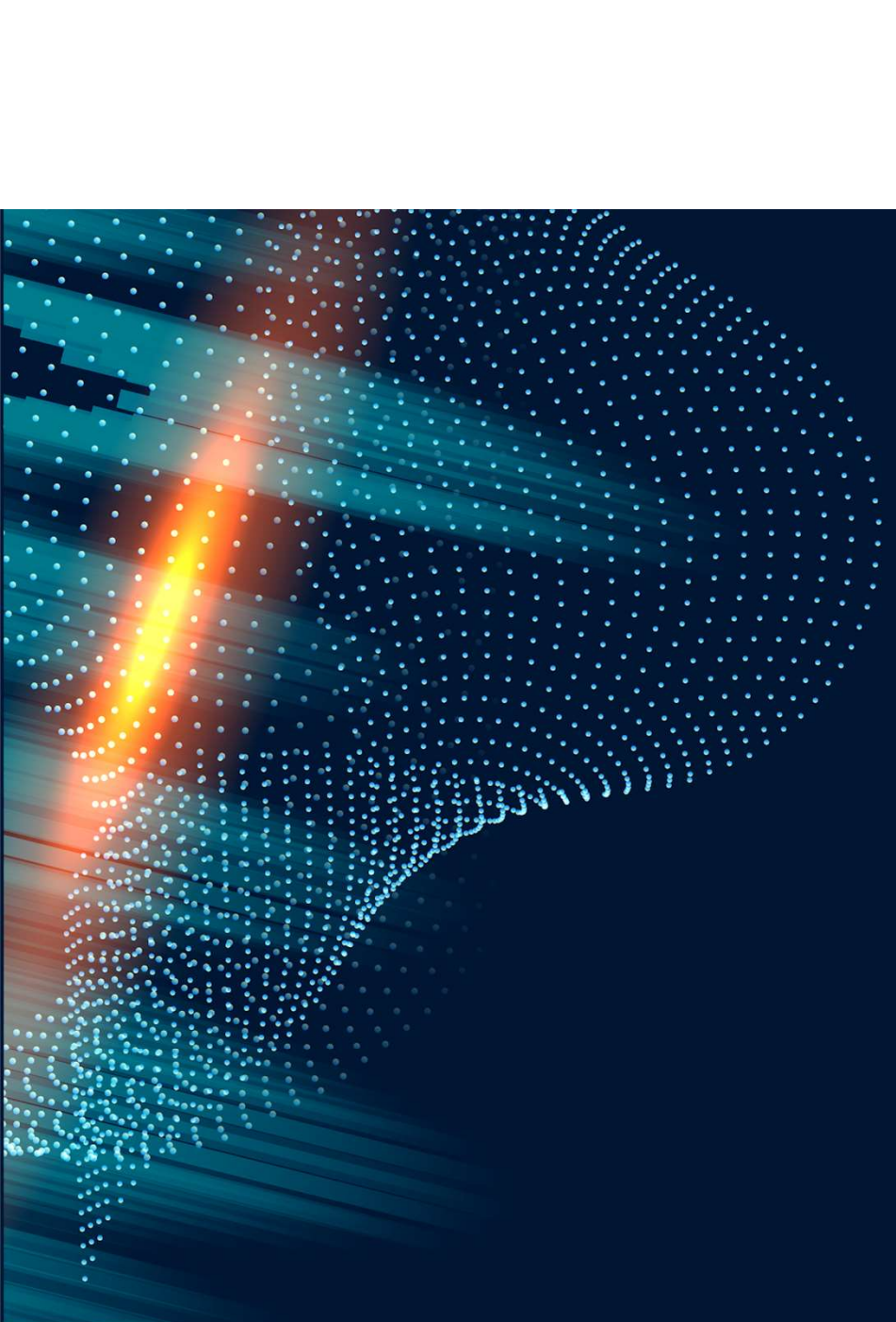
- **L'algoritmo di Dijkstra è efficiente e versatile;**
- **Facilmente implementabile;**
- **Limiti:**
 - Non risolve grafi con cicli negativi
 - Può richiedere molta memoria per grandi grafi
 - Si concentra solo sul singolo percorso più breve, senza gestire facilmente la ricerca di più percorsi
- **Soluzioni alternative:**
- **È fondamentale valutare i requisiti specifici e le alternative.**

«Fino ad oggi, i manager idioti del software misurano "la produttività dei programmatori" in termini di "linee di codice prodotte", mentre il concetto di "linee di codice trascorso" è molto più appropriato.»



Edsger Wybe Dijkstra





Grazie per l'attenzione!

Realizzato da: Scarale S. - Puzzolante;
Animazioni: Scarale S. - Puzzolante;
Lavoro: Puzzolante - Scarale S. - Bisceglia -
De Bonis - Cocomazzi - Tancredi - Cisternino
- Lauriola - Vigilante.